MINI PROJECT REPORT

VOTING SYSTEM

USING ETHEREUM SMART CONTRACT

ISE-II: Ethereum Transaction Setup and Execution

Submitted by

Nakshatra Sandip Pawar

Roll No.: 46

Department of Computer Science and Engineering

Academic Year: 2026–2027

Submitted to:

Mrs.Nandini N Patil

1. Aim

To develop and test a Solidity smart contract for a voting system that allows users to vote for registered candidates, maintains vote counts on the blockchain, and prevents the same wallet address from voting more than once.

2. Objectives

- Use Remix IDE to write, compile, deploy, and test a Solidity smart contract.
- Create candidate records containing candidate ID, name, and vote count.
- Allow a wallet address to cast a vote for a valid candidate.
- Prevent duplicate voting from the same wallet address.
- Retrieve candidate information and verify vote counts.
- Observe successful transactions, reverted transactions, and the difference between read-only calls and state-changing transactions.

3. Software and Tools Used

Tool	Purpose	Use in this project
Remix IDE	Online Solidity development environment	Used to write, compile, deploy, and test the smart contract.
Solidity	Smart contract programming language	Implements candidate storage, voting, vote counting, and duplicate-vote prevention.
Remix VM	Local simulated Ethereum environment	Provides test accounts and a simulated blockchain without real cryptocurrency.
MetaMask (optional)	Browser wallet	Can connect Remix to a supported test network for wallet-based deployment and transactions.
Ethereum test network (optional)	Public blockchain test environment	Can be used for a public testnet demonstration if required by the instructor.

4. Theory

A blockchain is a distributed ledger that records transactions in blocks. A smart contract is a program deployed on a blockchain and executed according to its code. This project uses a Solidity contract to store candidate details and maintain vote counts.

Each candidate is represented by a structure containing an ID, name, and vote count. A mapping stores candidate records by ID. Another mapping records whether a wallet address has already voted. When a vote is submitted, the contract checks that the address has not voted before and that the candidate ID is valid. If both conditions are satisfied, the address is marked as having voted and the selected candidate's vote count is increased.

This is an educational demonstration. It does not verify a voter's real-world identity, and the candidate-creation function in this version is publicly callable. Therefore, it should not be used for real elections without stronger access control, identity checks, and election-specific safeguards.

5. Smart Contract Source Code

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
```

```
contract VotingSystem {
    struct Candidate {
        uint id;
        string name;
        uint voteCount;
    }

    mapping(uint => Candidate) public candidates;
    mapping(address => bool) public hasVoted;

    uint public candidatesCount;

    constructor() {
        addCandidate("Candidate A");
        addCandidate("Candidate B");
        addCandidate("Candidate C");
    }

    function addCandidate(string memory _name) public {
        candidatesCount++;

        candidates[candidatesCount] = Candidate(
            candidatesCount,
            _name,
            0
        );
    }

    function vote(uint _candidateId) public {
        require(!hasVoted[msg.sender], "You have already voted");
        require(
            _candidateId > 0 && _candidateId <= candidatesCount,
            "Invalid candidate ID"
        );

        hasVoted[msg.sender] = true;
        candidates[_candidateId].voteCount++;
    }

    function getCandidate(uint _candidateId)
        public
        view
        returns (uint, string memory, uint)
    {
        require(
            _candidateId > 0 && _candidateId <= candidatesCount,
            "Invalid candidate ID"
        );

        Candidate memory c = candidates[_candidateId];
        return (c.id, c.name, c.voteCount);
    }
}
```

6. Explanation of the Contract

Component	Description
Candidate structure	Stores the candidate ID, name, and current vote count.
candidates mapping	Maps each candidate ID to its candidate record.
hasVoted mapping	Tracks whether a wallet address has already cast a vote.
candidatesCount	Stores the total number of candidates currently recorded.
constructor()	Creates Candidate A, Candidate B, and Candidate C when the contract is deployed.
addCandidate()	Adds a candidate and assigns the next candidate ID.
vote()	Checks duplicate voting and candidate ID validity, then increments the vote count.
getCandidate()	A read-only function that returns a candidate's ID, name, and vote count.

7. Procedure

1. Open Remix IDE at https://remix.ethereum.org/.
2. In File Explorer, create a Solidity file named VotingSystem.sol.
3. Paste the smart contract source code into the file and save it.
4. Open Solidity Compiler and select a compatible Solidity 0.8.x compiler version.
5. Compile VotingSystem.sol and confirm that compilation succeeds.
6. Open Deploy & Run Transactions. Select Remix VM for local testing.
7. Select VotingSystem in the contract dropdown and click Deploy.
8. Expand the deployed contract and call candidatesCount. The expected initial result is 3.
9. Call getCandidate with ID 1. The expected initial result is Candidate A with 0 votes.
10. Call vote with candidate ID 1. The transaction should succeed if the selected account has not voted before.
11. Call getCandidate with ID 1 again and verify that the vote count has increased by one.
12. Call vote again from the same account. The transaction should revert with the message 'You have already voted'.
13. Switch to a different Remix VM account and call vote with candidate ID 2. This vote should succeed for the new address.
14. Call getCandidate with IDs 1, 2, and 3 to inspect the vote counts.
15. Capture screenshots of the source code, successful compilation, deployment, successful vote, duplicate-vote rejection, and final candidate counts.
16. If MetaMask and a public test network are required, connect Remix to the chosen test network and deploy the contract again. Record only the actual transaction details observed.

8. Transaction Details and Observations

Complete this table using the actual values shown in Remix, MetaMask, or a blockchain explorer. For Remix VM, record the local transaction details available in the interface. Do not enter invented values.

Parameter	Observed value
Execution environment / network	Remix VM
Deployed contract address	0xd8b934580fcE35a11B58C6D73aDeE468a2833fa8
Deployment transaction status	complete
Deployment transaction hash (if available)	0x8a53c58127035217a55e2da02f125e9b447030b10d8a3702c92545dd46d0f42e
Initial candidatesCount	3
Candidate A vote result	1
Candidate B vote result	1
Duplicate vote test	'You have already voted'

9. Transaction Analysis

9.1 Steps involved in transaction execution

The user invokes vote() in Remix. The selected account submits the transaction to the execution environment. The contract checks the caller's voting status and the candidate ID. If the checks pass, the contract marks the address as having voted and increments the selected candidate's count. If a check fails, the transaction reverts and its state changes are not applied.

9.2 Read-only calls versus state-changing transactions

Feature	Read-only call	State-changing transaction
Example	getCandidate(), candidatesCount	vote(), addCandidate()
Purpose	Reads stored information	Updates contract state
Changes state	No	Yes, if successful
Gas / transaction	Normally no on-chain transaction fee for a read-only call	Consumes gas and requires a transaction
Wallet approval	Normally not required for a read-only call	Required when using a wallet to submit a transaction

9.3 Role of gas

Gas measures the computational work required to execute operations on Ethereum-compatible networks. State-changing operations such as voting consume gas. In Remix VM, testing uses a simulated environment and does not require real ETH.

9.4 Duplicate-vote prevention

The hasVoted mapping associates each wallet address with a Boolean value. Before recording a vote, the contract requires that this value is false. After a successful vote, the value becomes true. A second attempt from the same address reverts with 'You have already voted'.

9.5 Transaction reversion

A reverted transaction means the contract's require condition was not satisfied. In this experiment, a duplicate-vote revert is the expected result when the same account tries to vote twice. It is not, by itself, evidence of an out-of-gas problem.

10. Screenshots

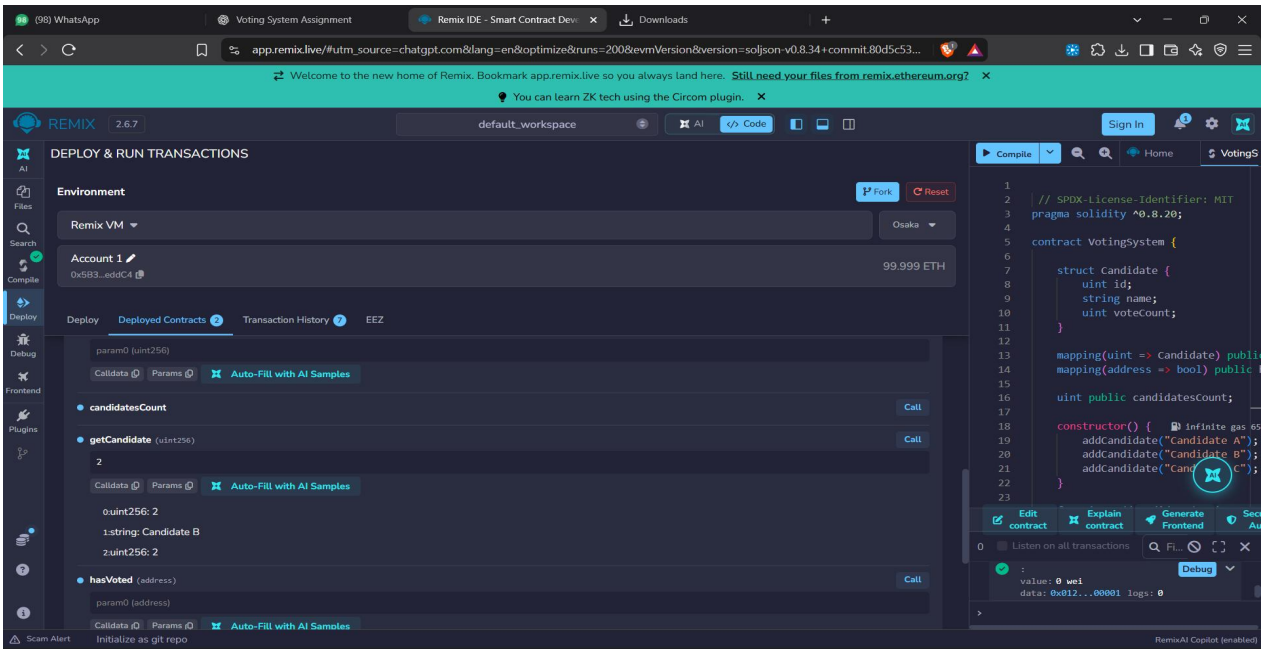


Fig. 1 Deploy

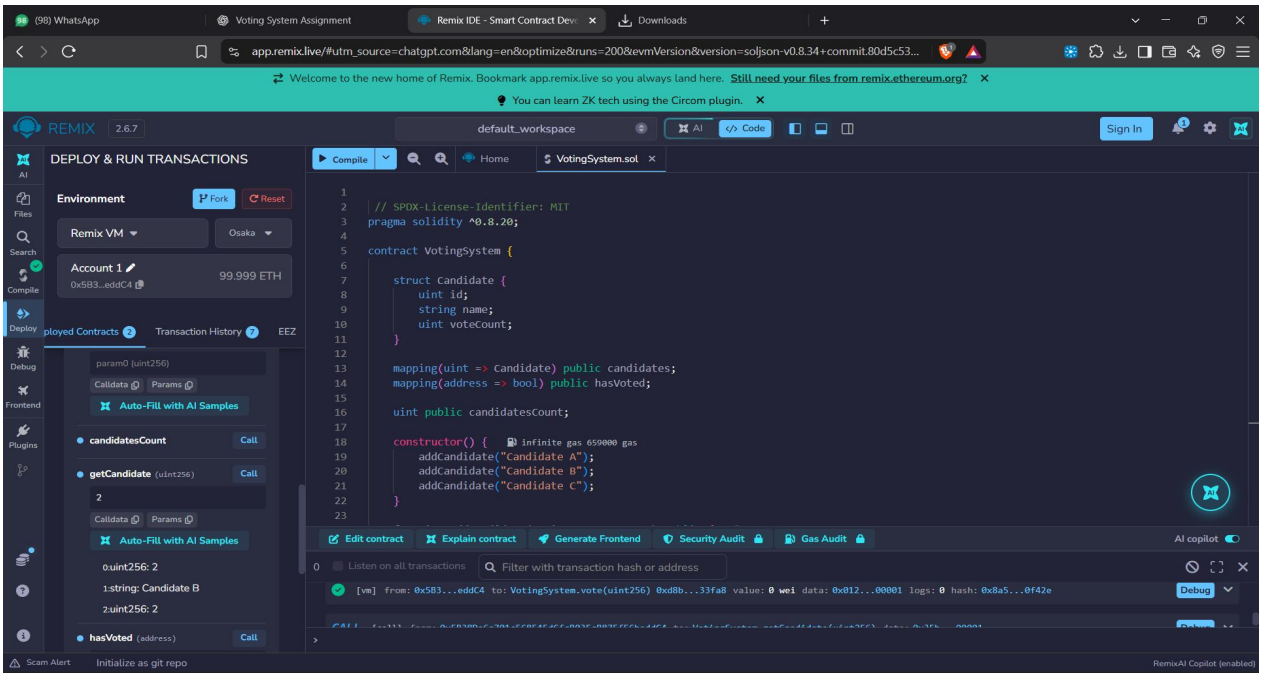


Fig. 2 Compile

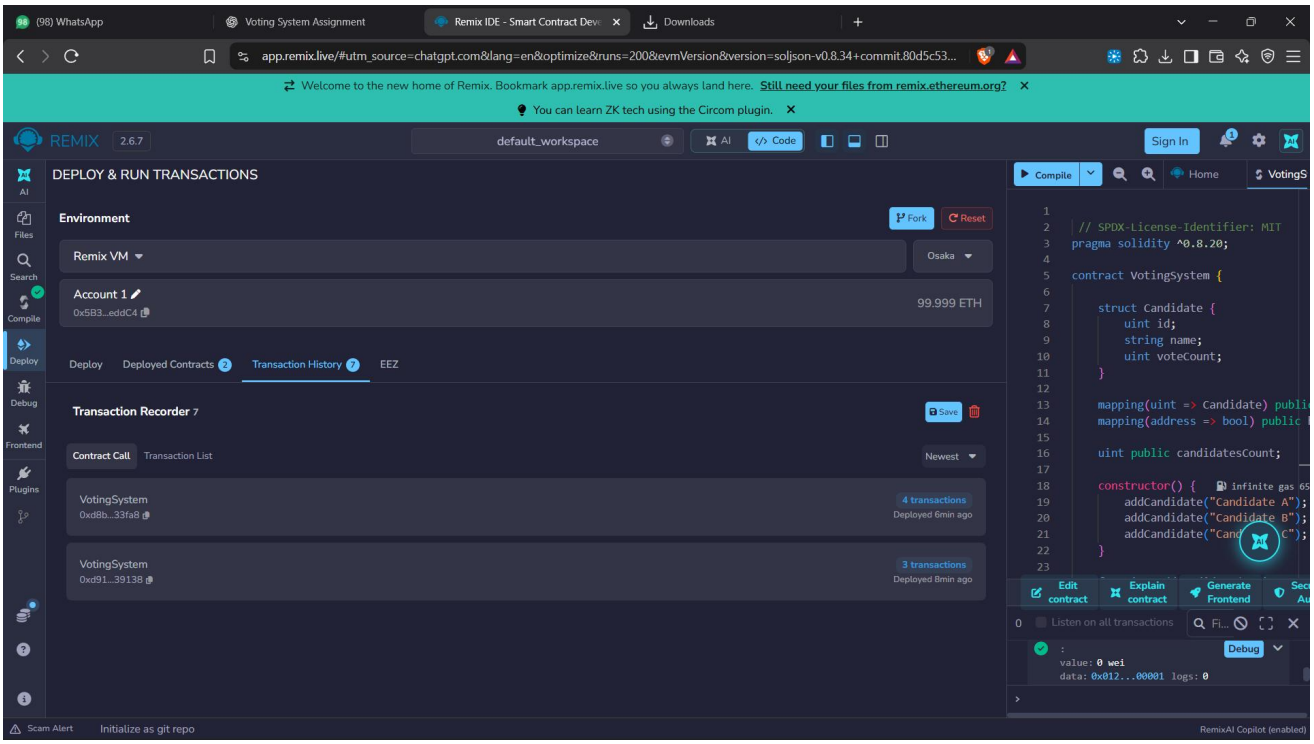


Fig. 3 Transaction

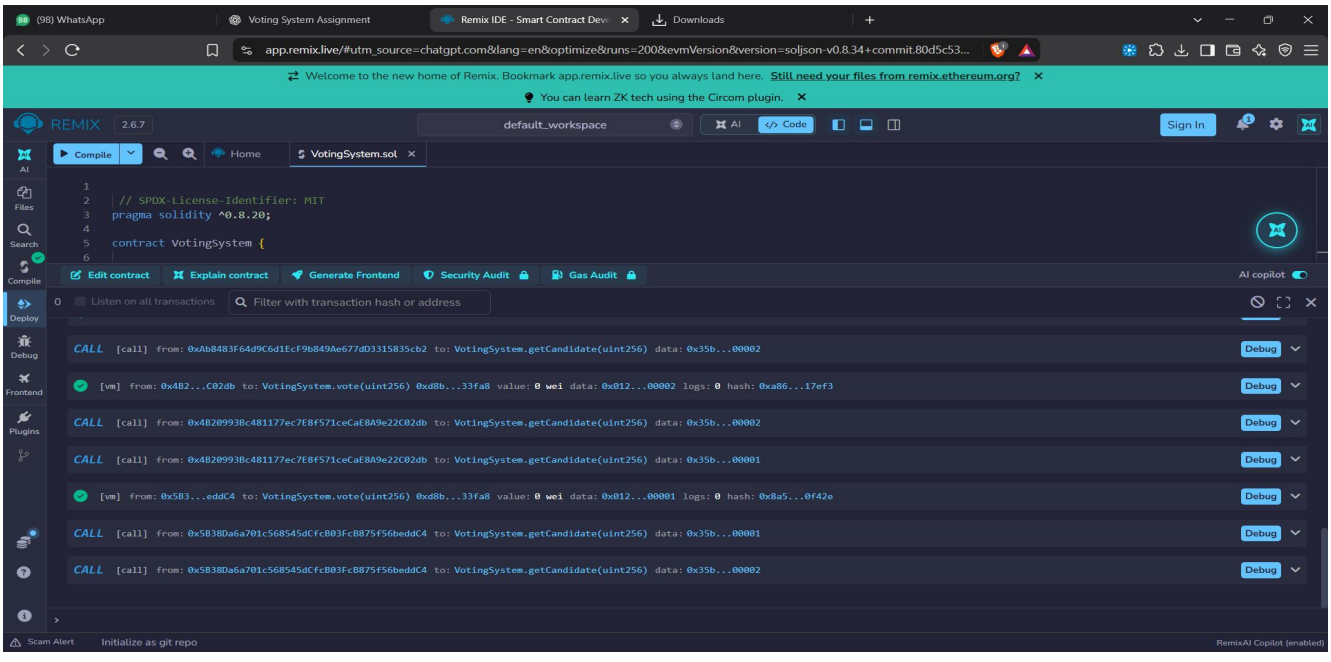


Fig. 4 Success

11. Result

The VotingSystem smart contract is designed to create candidate records, record votes, maintain vote counts, and reject repeated voting from the same wallet address. After executing the experiment, record the actual outcome here: compilation [successful/unsuccessful], deployment [successful/unsuccessful], first vote [successful/unsuccessful], duplicate vote [reverted/other], and final candidate vote counts. Do not mark the experiment successful until these results have been observed.

12. Conclusion

This mini-project demonstrates how a Solidity smart contract can record votes and maintain candidate vote counts in a blockchain execution environment. It uses a mapping to track whether an address has voted and validates candidate IDs before updating results. The experiment also demonstrates successful state-changing transactions, read-only calls, and expected transaction reversion for duplicate voting. The contract is intended for educational testing and would require stronger access control, voter identity checks, and additional safeguards before any real election use.

13. References

- Remix IDE: https://remix.ethereum.org/
- Solidity Documentation: https://docs.soliditylang.org/
- Ethereum Developer Documentation: https://ethereum.org/en/developers/docs/
- MetaMask (optional wallet): https://metamask.io/